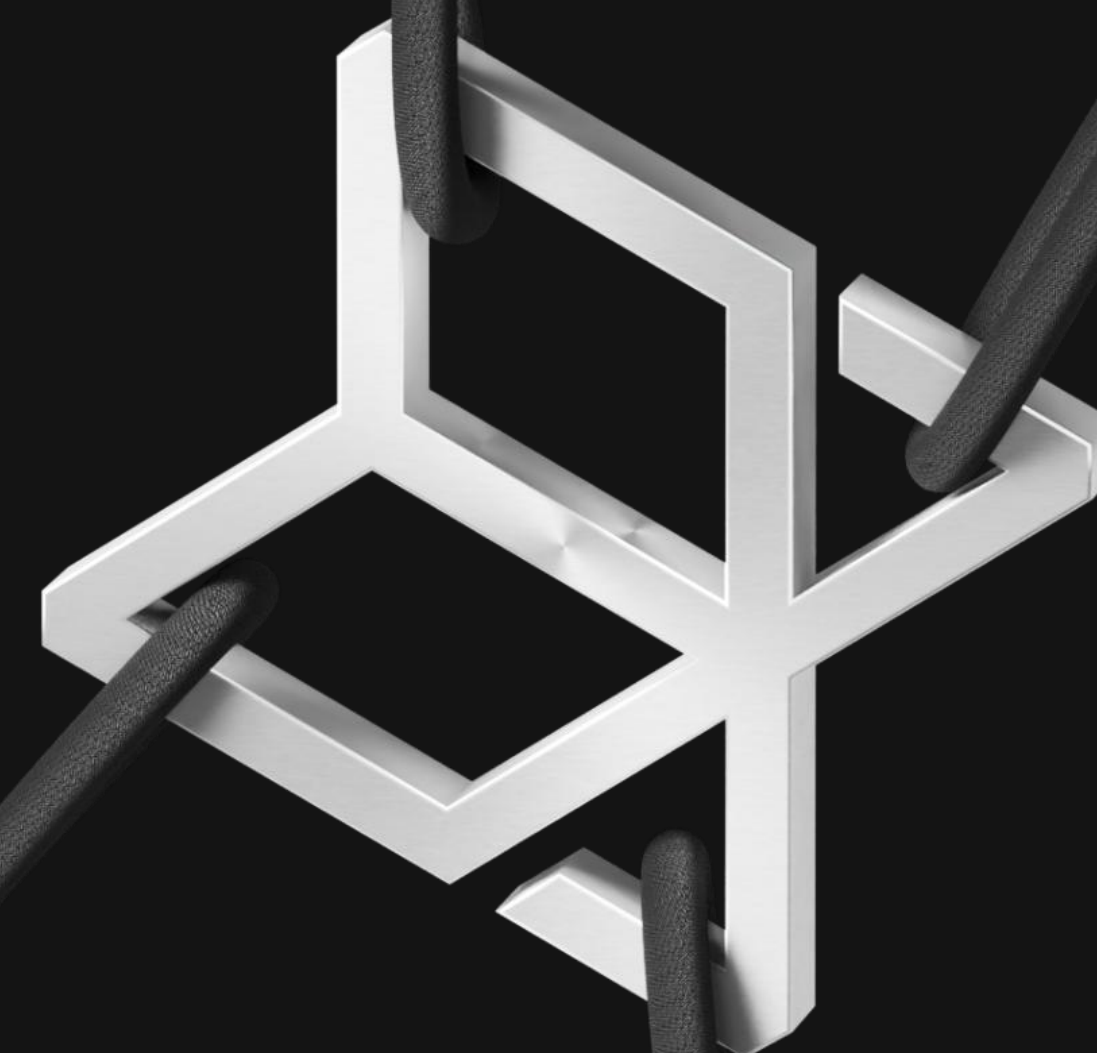


Масштабирование

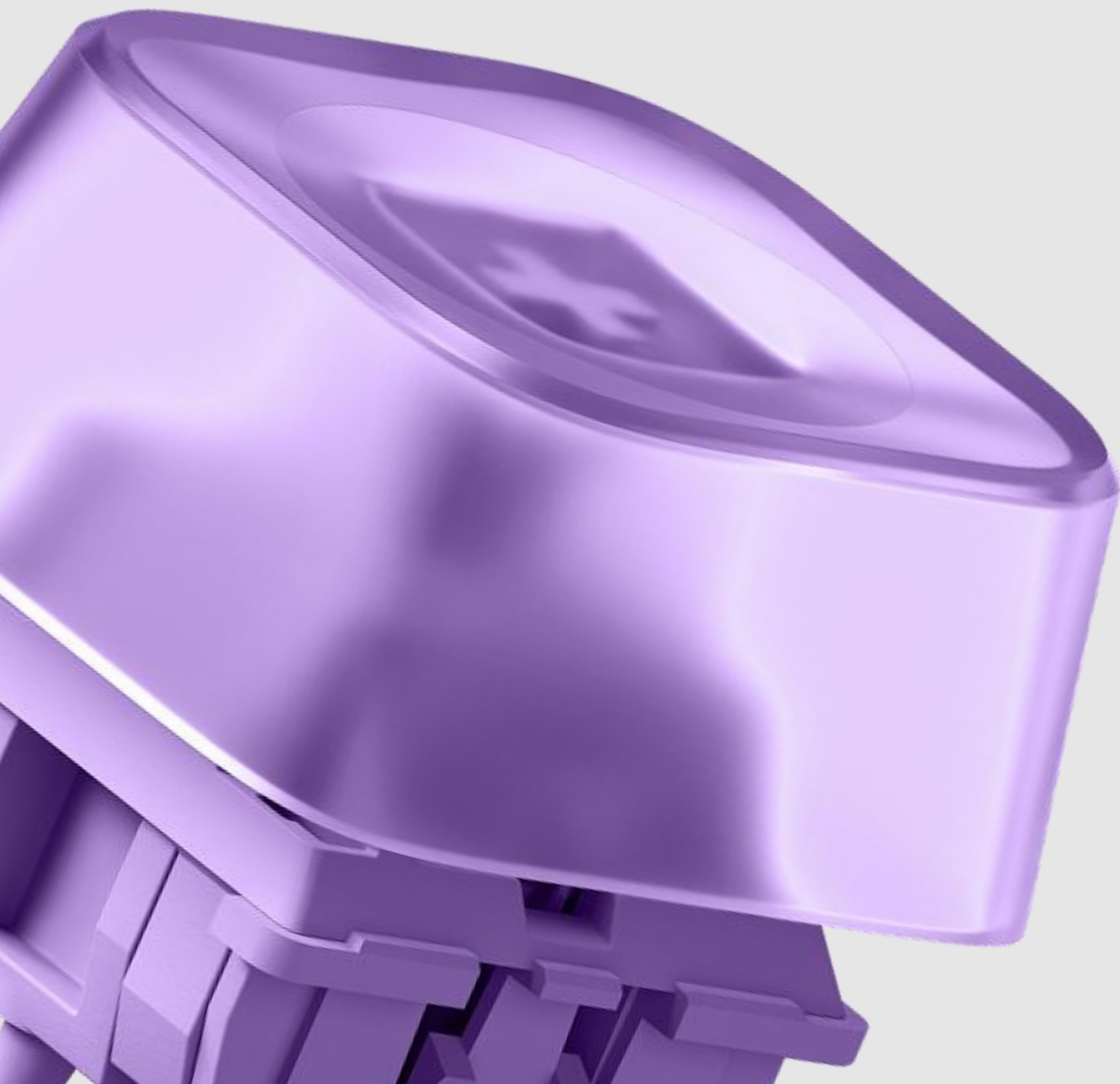
Резервное копирование и репликация

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 15



О чём сегодня поговорим



→ Проблема масштаба

→ Вертикальное масштабирование

→ Горизонтальное масштабирование

→ Партицирование

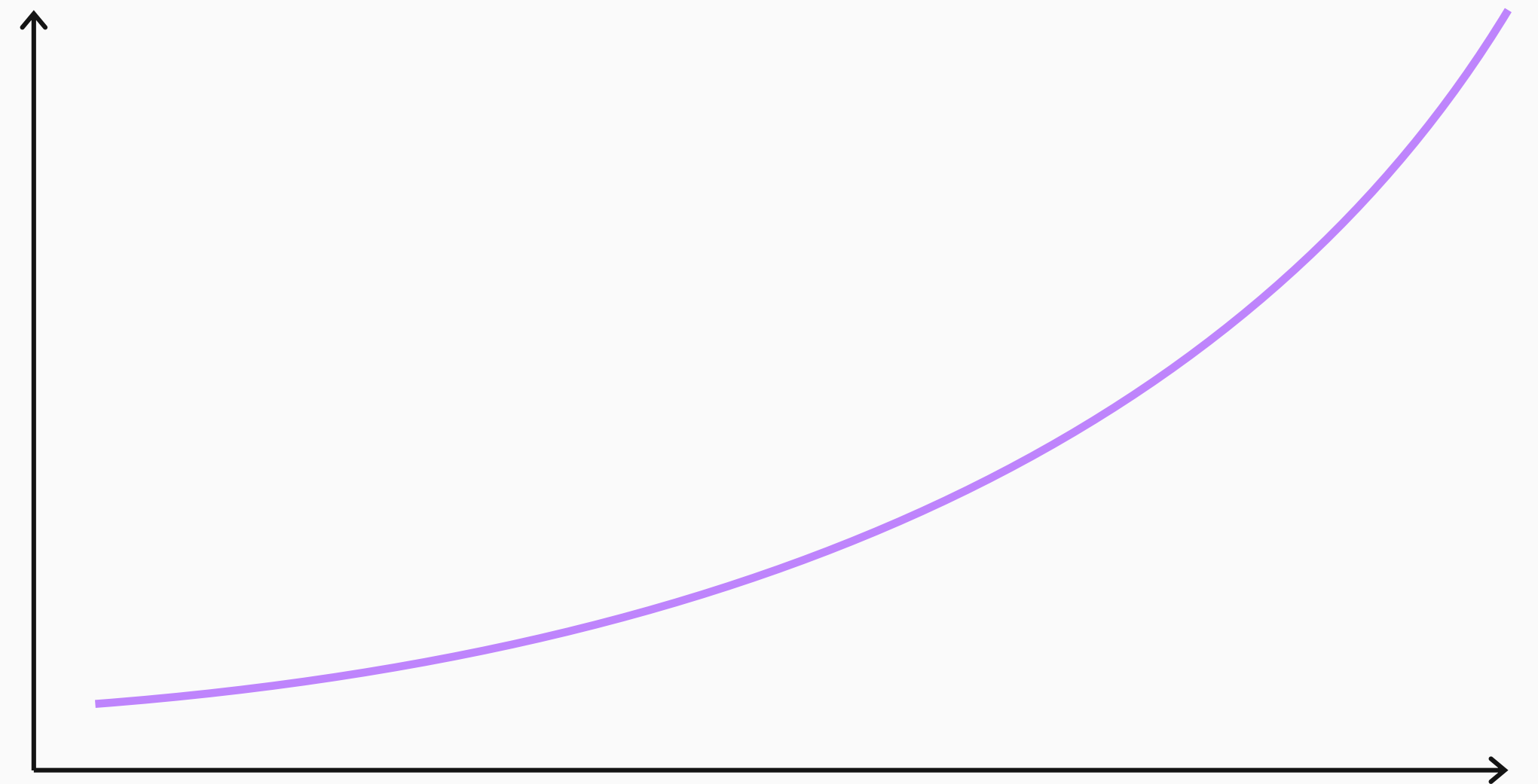
→ Стекланный потолок

Что делать, когда нагрузка растёт?

ПРОБЛЕМЫ

- CPU 100%
- RAM занята
- I/O перегружен
- Запросы медленные

Как нам увеличить «мощность» нашей базы данных?



Вертикальное масштабирование

ПУТЬ №1

ПОКУПАЕМ СЕРВЕР ПОБОЛЬШЕ

Преимущества



- Просто
(не требует изменений в коде)
- Эффективно
(до поры до времени)

Недостатки

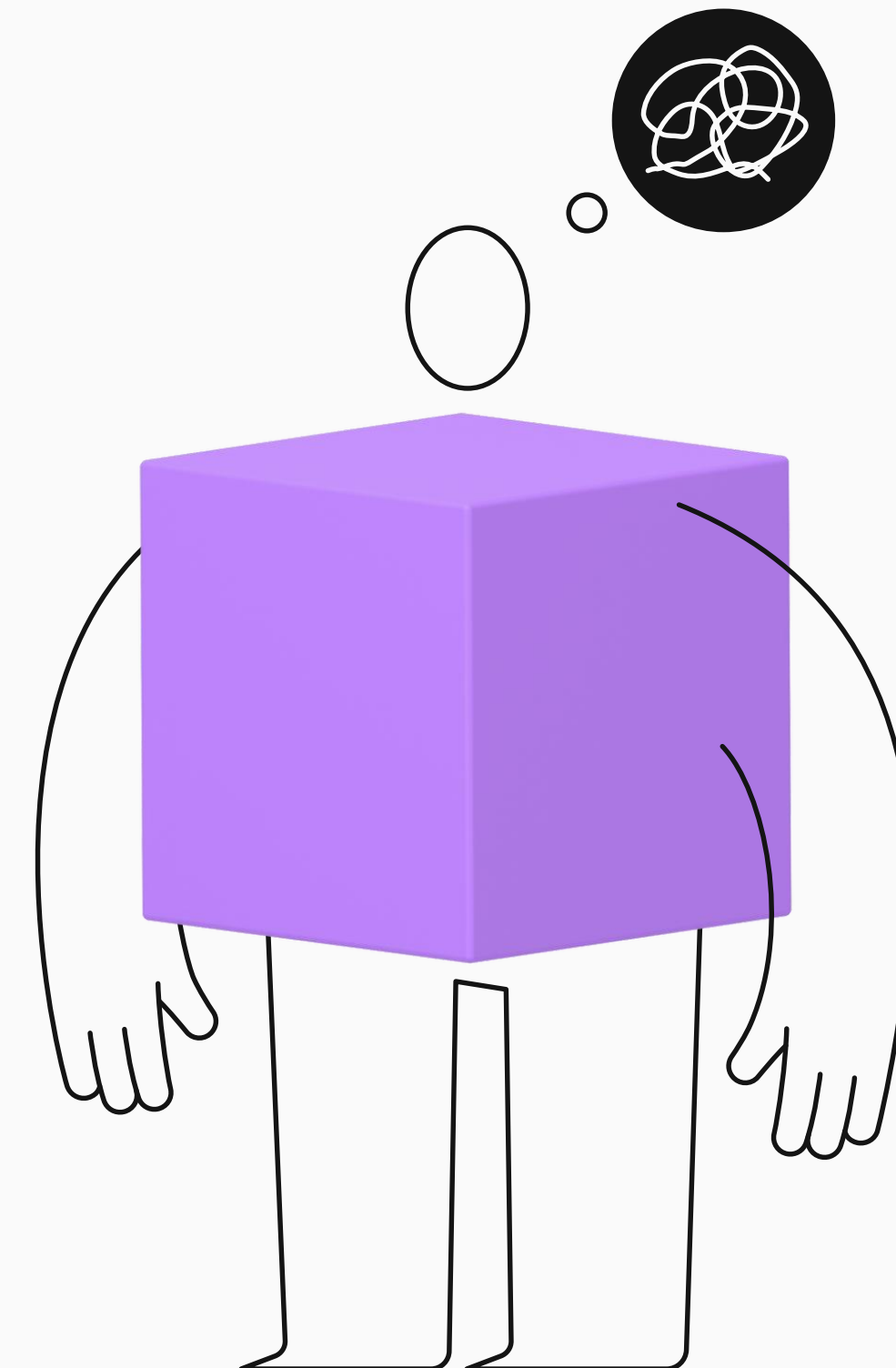


- Очень дорого
- Есть физический предел
- Всё ещё «единая точка отказа»

Почему Scale-Up так хорошо работает (сначала)

Единый сервер,

- на котором единый shared_buffers (вся горячая память в одном месте);
- единый WAL-писатель;
- единый пул соединений;
- процессы общаются через быструю общую память.



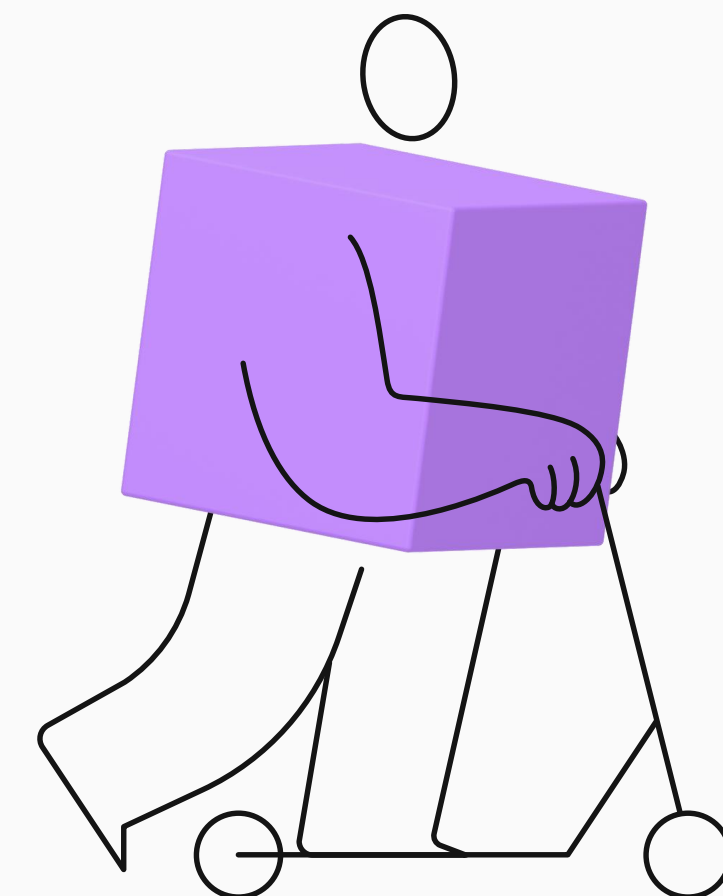
Один сервер — это узкое горлышко и точка отказа

Масштабируемость чтения

Один сервер может не справляться с большим количеством SELECT-запросов от аналитиков и пользователей.

Высокая доступность

Если единственный сервер падает, сервис полностью останавливается на время восстановления из бэкапа (что может занять часы).



Горизонтальное масштабирование

ПУТЬ №2

ПОКУПАЕМ МНОГО ДЕШЁВЫХ СЕРВЕРОВ

Преимущества



- Экономически эффективно
- Экономически эффективно
- Высокая отказоустойчивость

Недостатки



- СУБД должна уметь работать в распределённом режиме

Как PostgreSQL масштабируется горизонтально?

ОТВЕТ: С НЮАНСАМИ

Масштабирование чтения

- **(SELECT):** решение в PostgreSQL → репликация
- Легко масштабируется

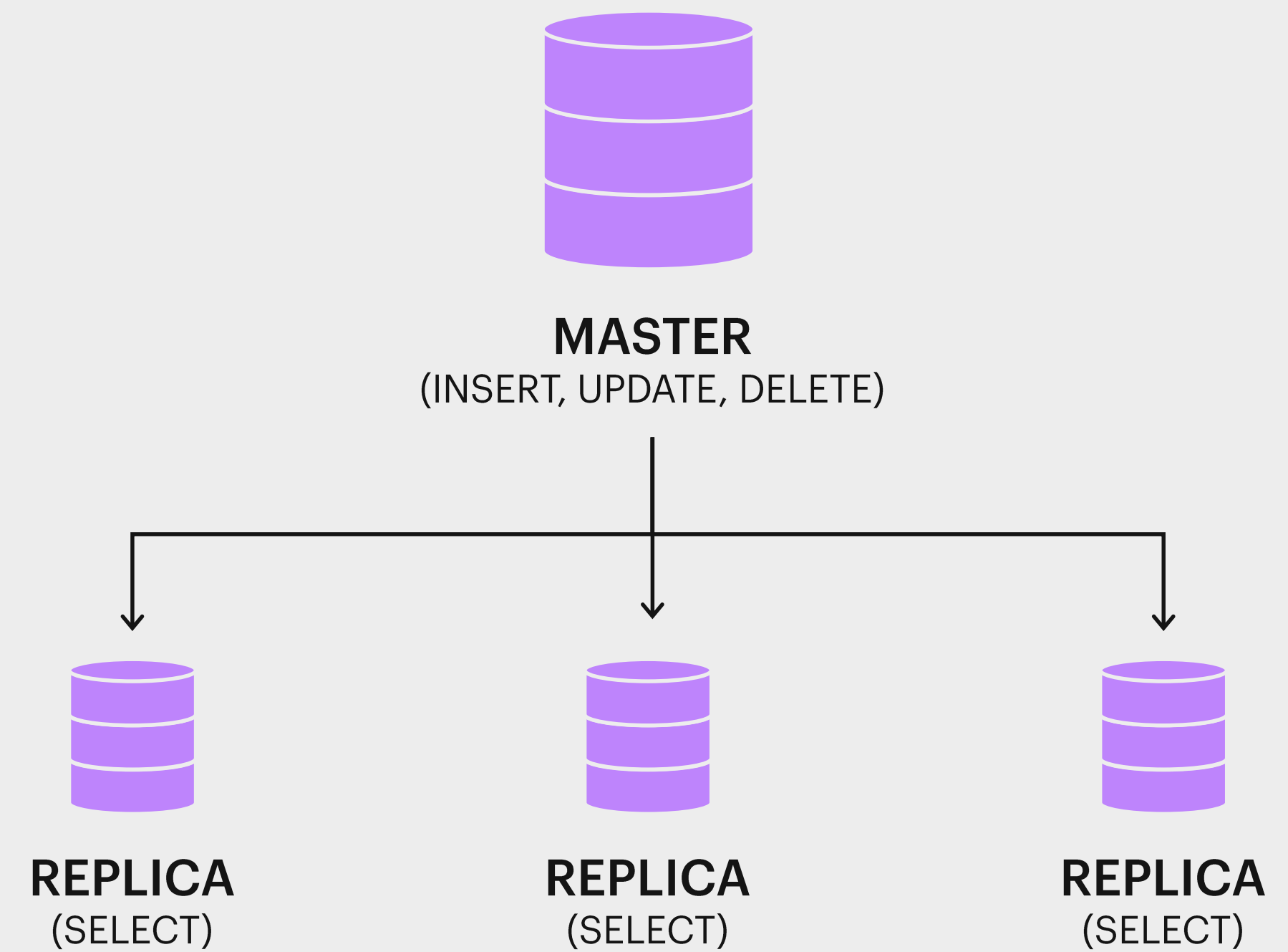
Масштабирование записи

- **(INSERT, UPDATE):** решение в PostgreSQL → гораздо сложнее
- «Из коробки» не масштабируется

Создаём «клонов» нашей базы

Преимущества

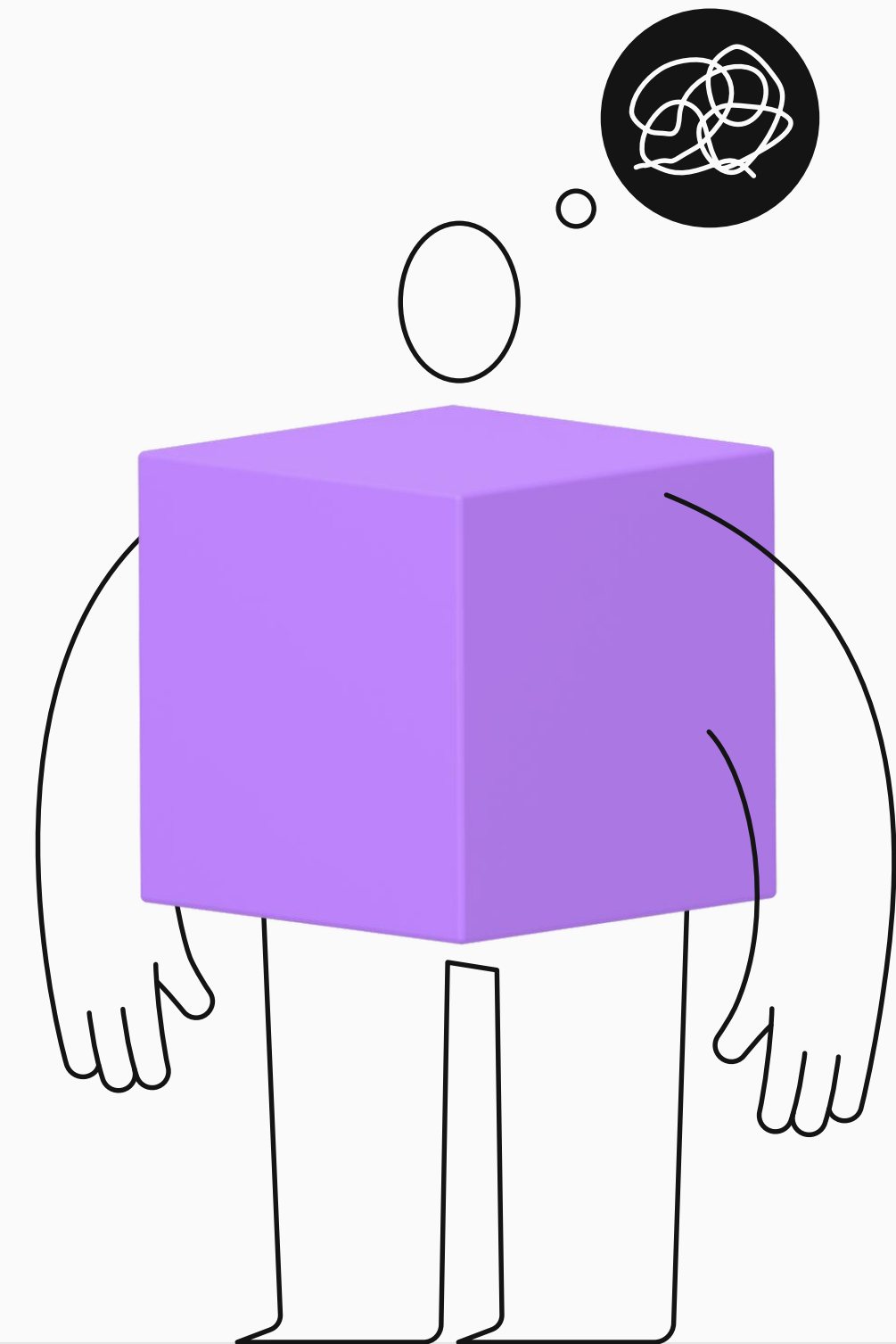
- Разгрузка мастера
- Отказоустойчивость



Как реплика узнаёт об изменениях?

Ключевые компоненты

- На мастере генерируется поток WAL-записей
- Этот поток в реальном времени отправляется по сети в реплику
- Реплика получает этот поток и применяет к своим файлам данных

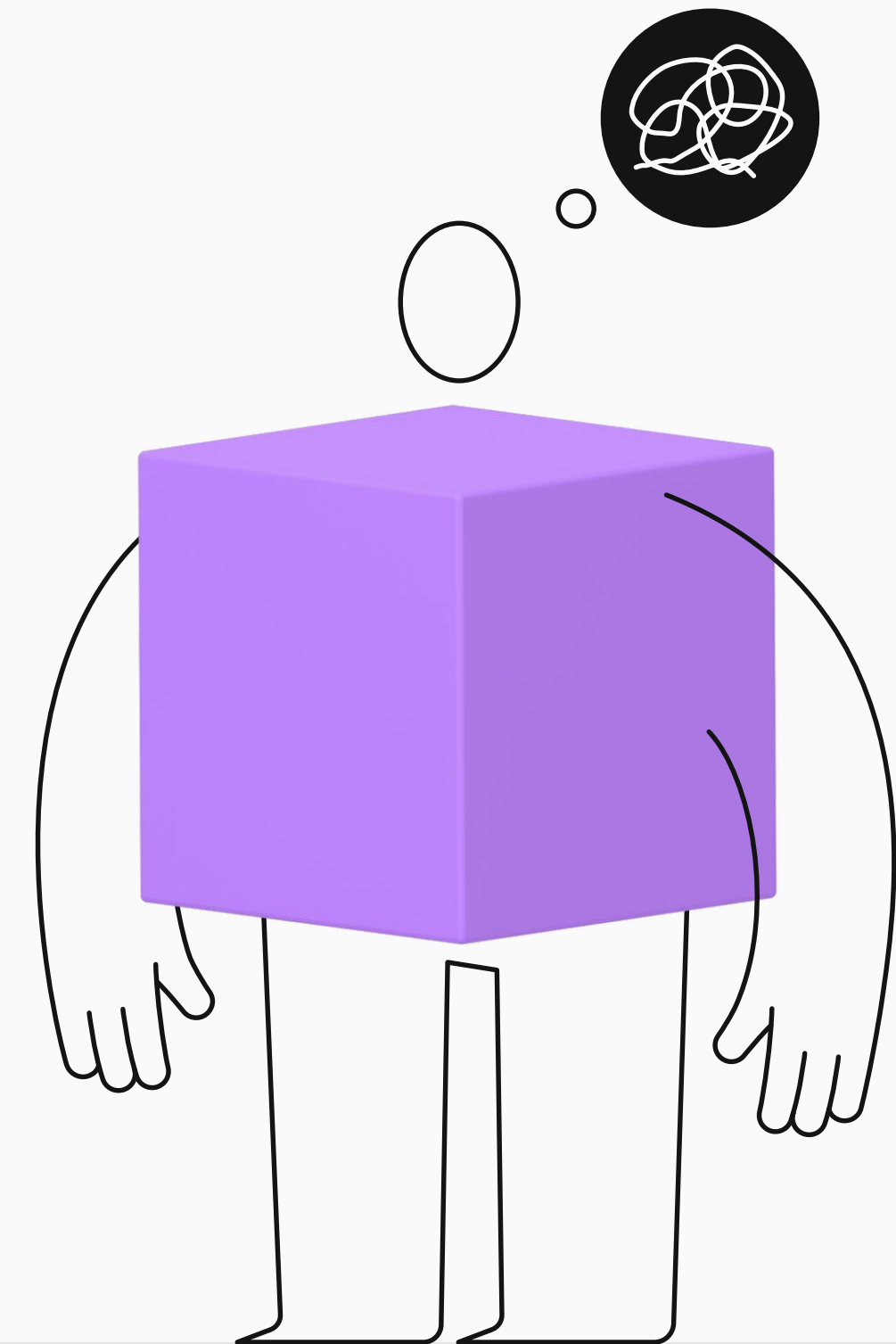


replication lag

Обратная сторона асинхронности

- 01 T1: на мастере выполняется INSERT.
- 02 T2: приложение получает COMMIT OK.
- 03 T3: приложение сразу же делает SELECT на реплику.
- 04 T4: WAL-запись ещё не «долетела» и не применилась.
- 05 Результат: реплика возвращает «не найдено».

Приложение видит несогласованные данные.



Компромисс между надёжностью и скоростью

	Асинхронная	Синхронная
Как работает	COMMIT на мастере завершается мгновенно	COMMIT на мастере ждёт подтверждения
Плюсы	Очень быстро	Гарантия нулевой потери
Минусы	Если мастер упал после COMMIT, может быть replication lag	Запись на мастере медленная

synchronous_commit и **synchronous_standby_names**

postgresql.conf

- **synchronous_commit = on**
(по умолчанию, ждёт только локальной записи WAL)
- **synchronous_commit = remote_write**
(мастер ждёт, пока реплика получила WAL, но ещё не записала)
- **synchronous_commit = remote_apply**
(самый надёжный, ждёт пока реплика и записала, и применила изменение)

synchronous_standby_names = 'ANY 1 (replica1, replica2)':
ждать подтверждения от **любой одной** из указанных реплик.

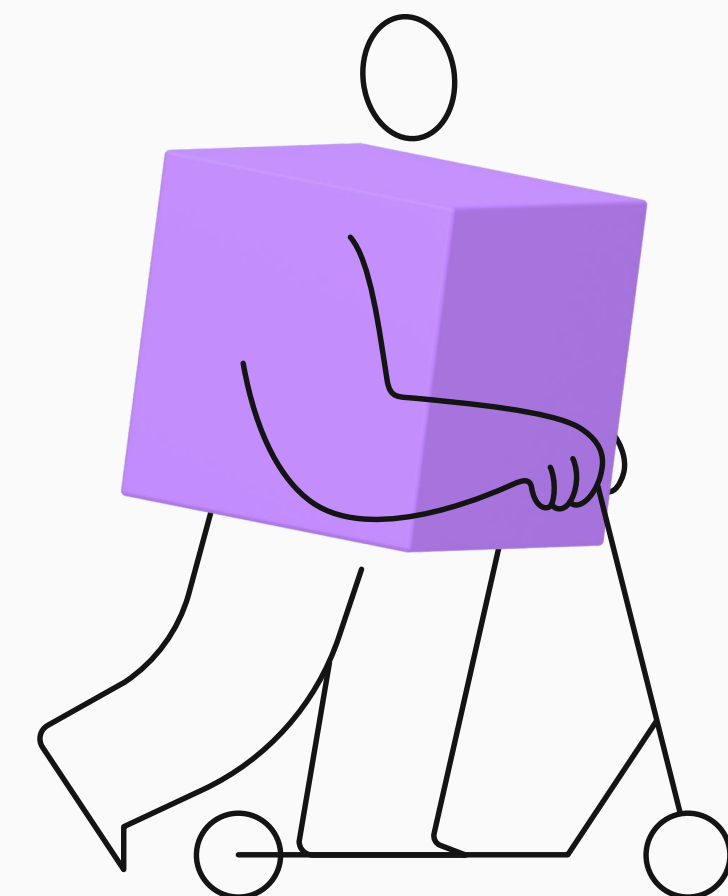
Реплицируем не «байты», а «смысл»

Логическая

- Декодирует WAL-лог события
- Можно реплицировать только часть
- Можно реплицировать в разные версии PG
- Можно реплицировать в другие СУБД

Физическая (потокковая)

- Копирует физические изменения (WAL)
- Реплика — точная побайтовая копия мастера
- Просто и надёжно



Когда выбирать логическую репликацию?

Сценарий 1: централизованный DWH

Несколько OLTP-баз (микросервисов) реплицируют свои ключевые таблицы в одну большую аналитическую базу.

01

Сценарий 2: «Нулевой даунтайм» при обновлении мажорной версии

Настраивается логическая репликация со старой версии на новую, а затем происходит переключение.

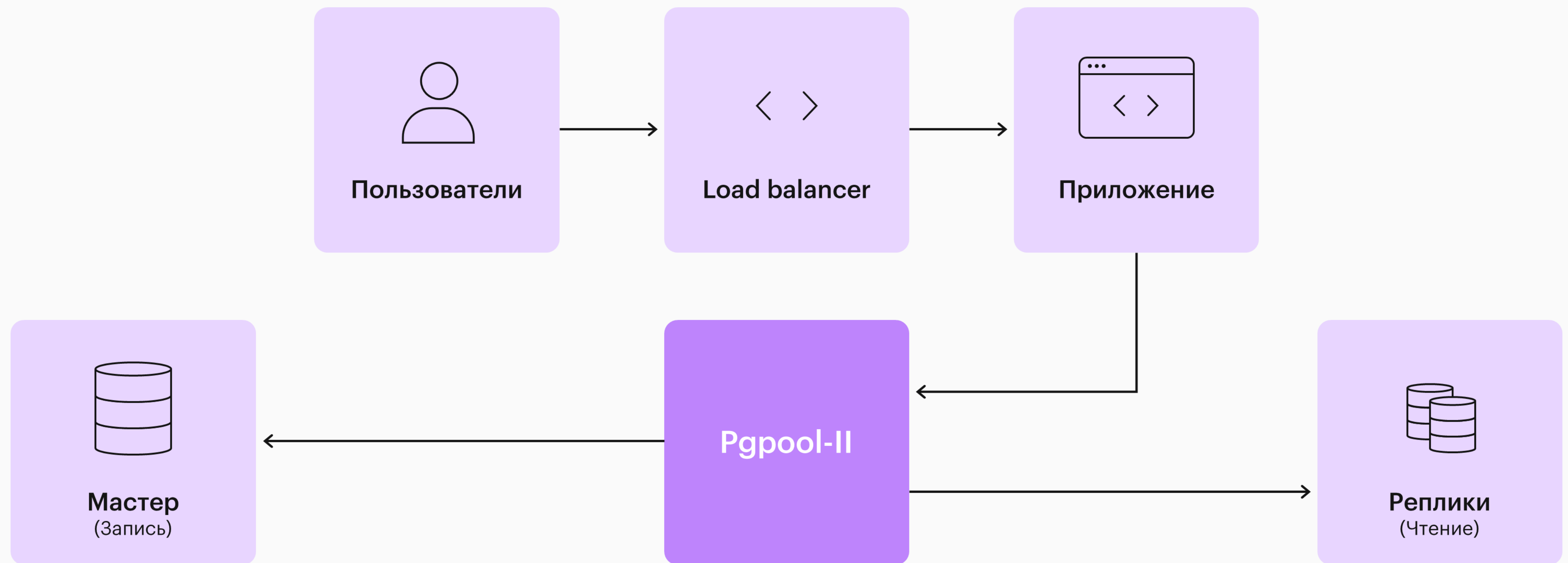
02

Сценарий 3: интеграция с Kafka

С помощью расширения wal2json можно превратить поток WAL в поток JSON-событий и отправить его в Kafka.

03

Архитектура с репликацией

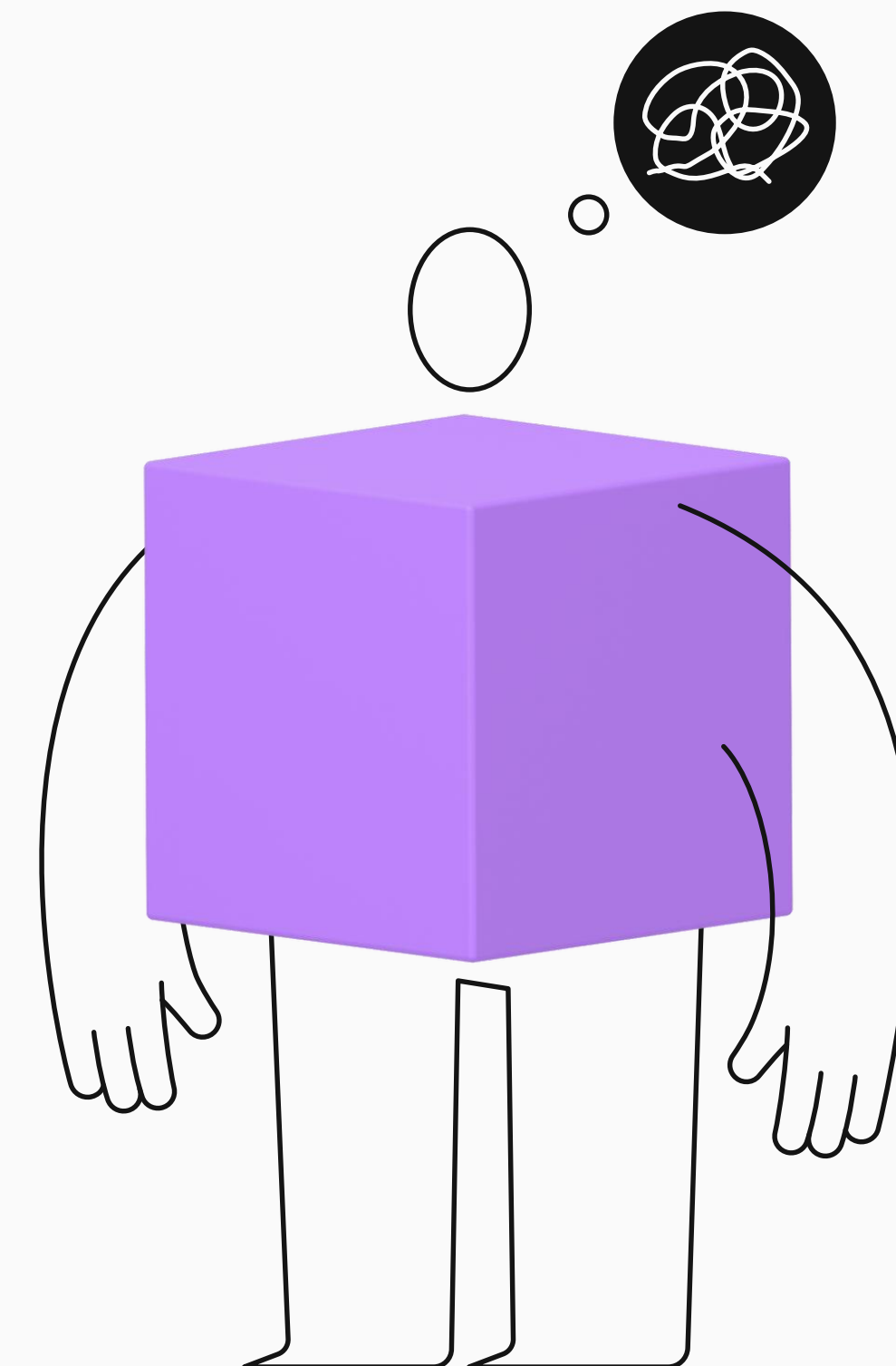


Гигантские таблицы

ПРОБЛЕМА

У нас в таблице миллиарды строк.

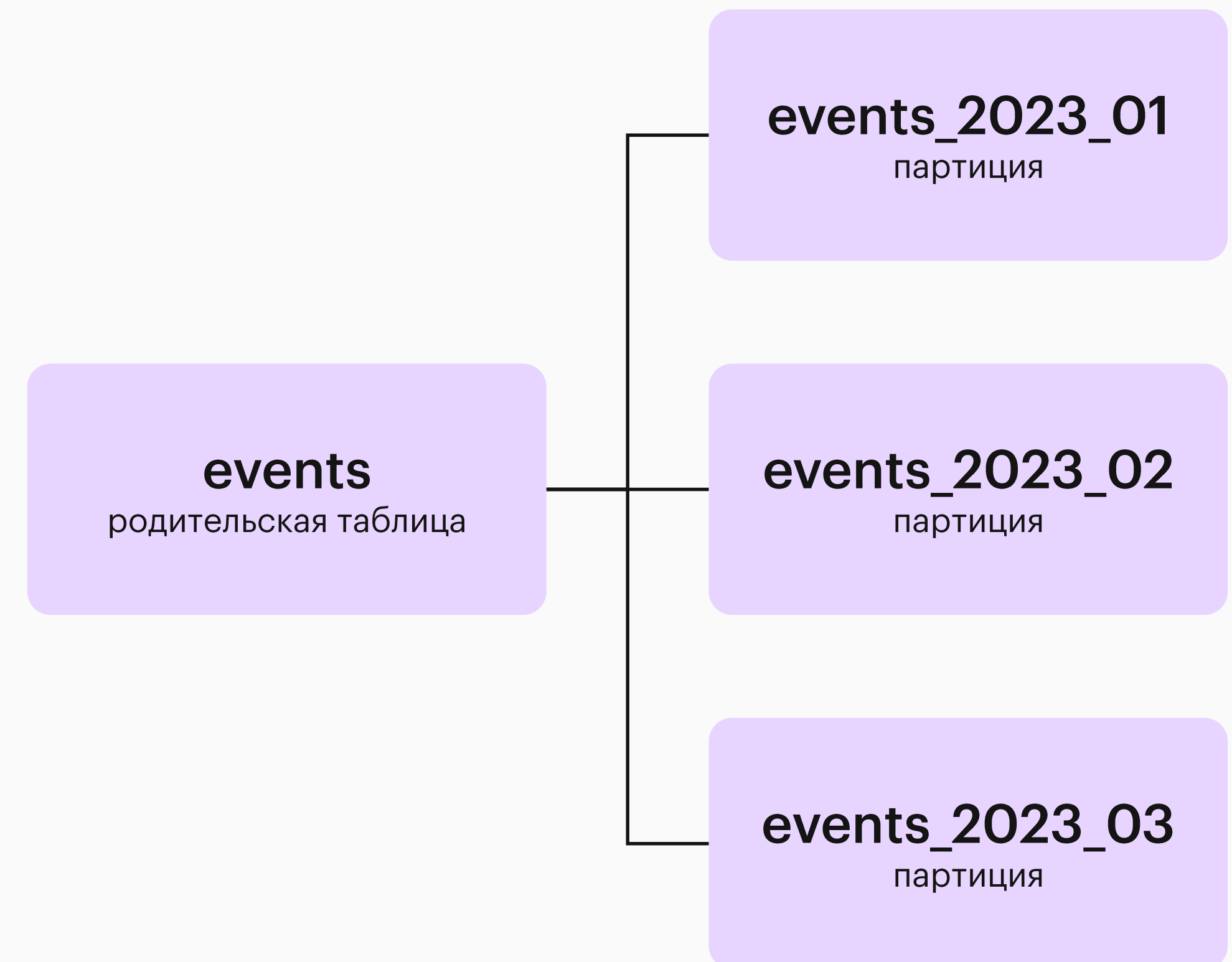
- SELECT становятся медленнее даже с индексами.
- VACUUM работает очень долго.
- Добавление индекса (CREATE INDEX) может занять часы.
- Удаление старых данных (DELETE) — очень дорогая операция.



Декларативное партицирование

РЕШЕНИЕ

- Вся работа идёт только с родительской таблицей.
- PostgreSQL автоматически перенаправляет строку в нужную партицию.
- При **SELECT ... WHERE date = '2023-01-15'**, PostgreSQL сканирует только одну партицию.



Типы партицирования

By Range

- **Ключ:** дата и число
- Распределение партиций по диапазону

By List

- **Ключ:** категория, регион, статус
- Определяет данные в партиции по списку

By Hash

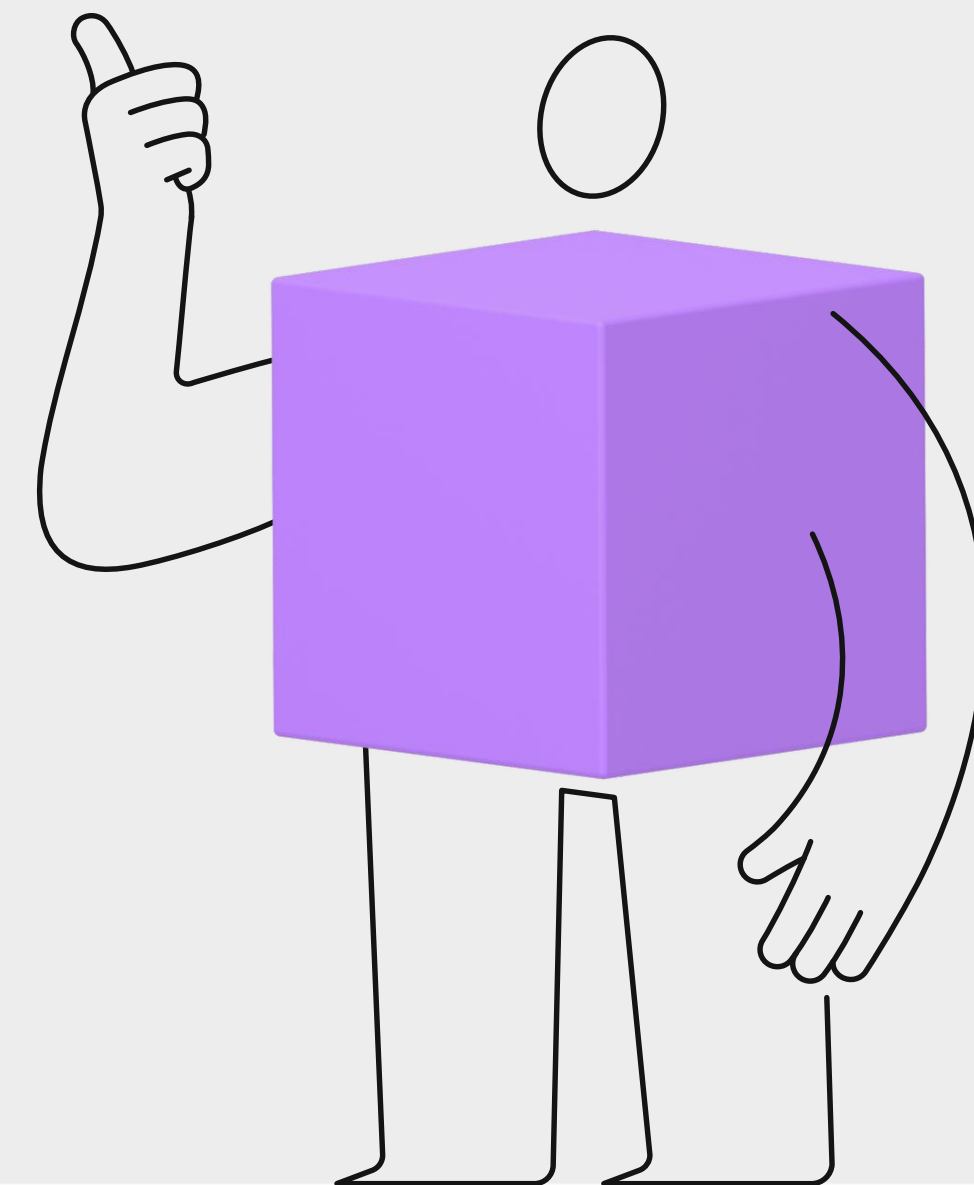
- **Ключ:** ID User, ID Client
- Равномерно распределяет данные по заданному количеству партиций

Преимущества партицирования

Преимущества



- Partition Pruning (отсечение партиций)
- Простое удаление старых данных
- Ускорение обслуживания



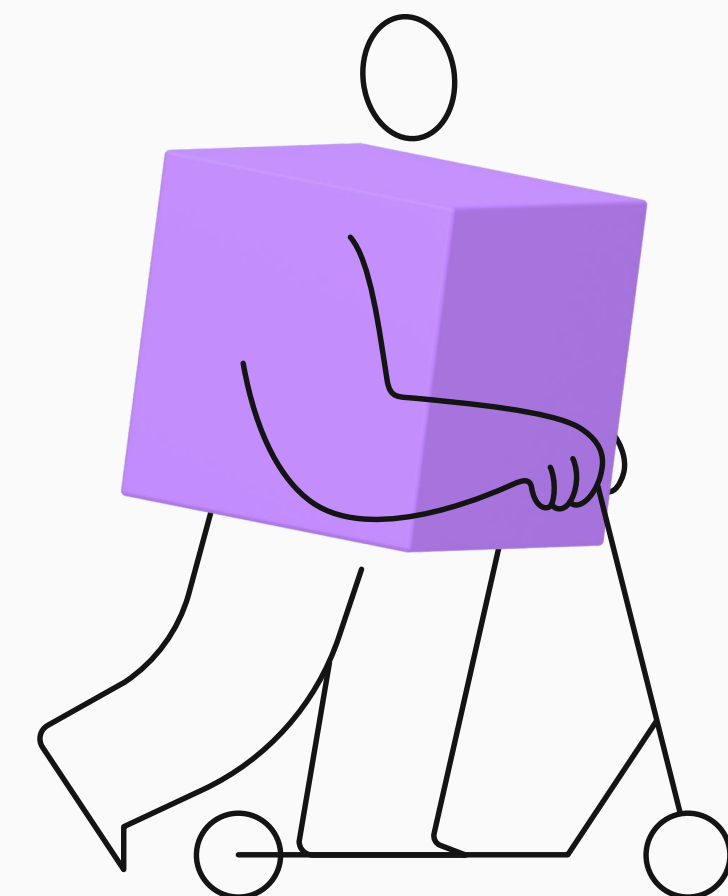
Пример DDL для партицирования по диапазону

Создание родительской таблицы

```
CREATE TABLE events ( ... ) PARTITION BY RANGE (created_at);
```

Создание родительской таблицы

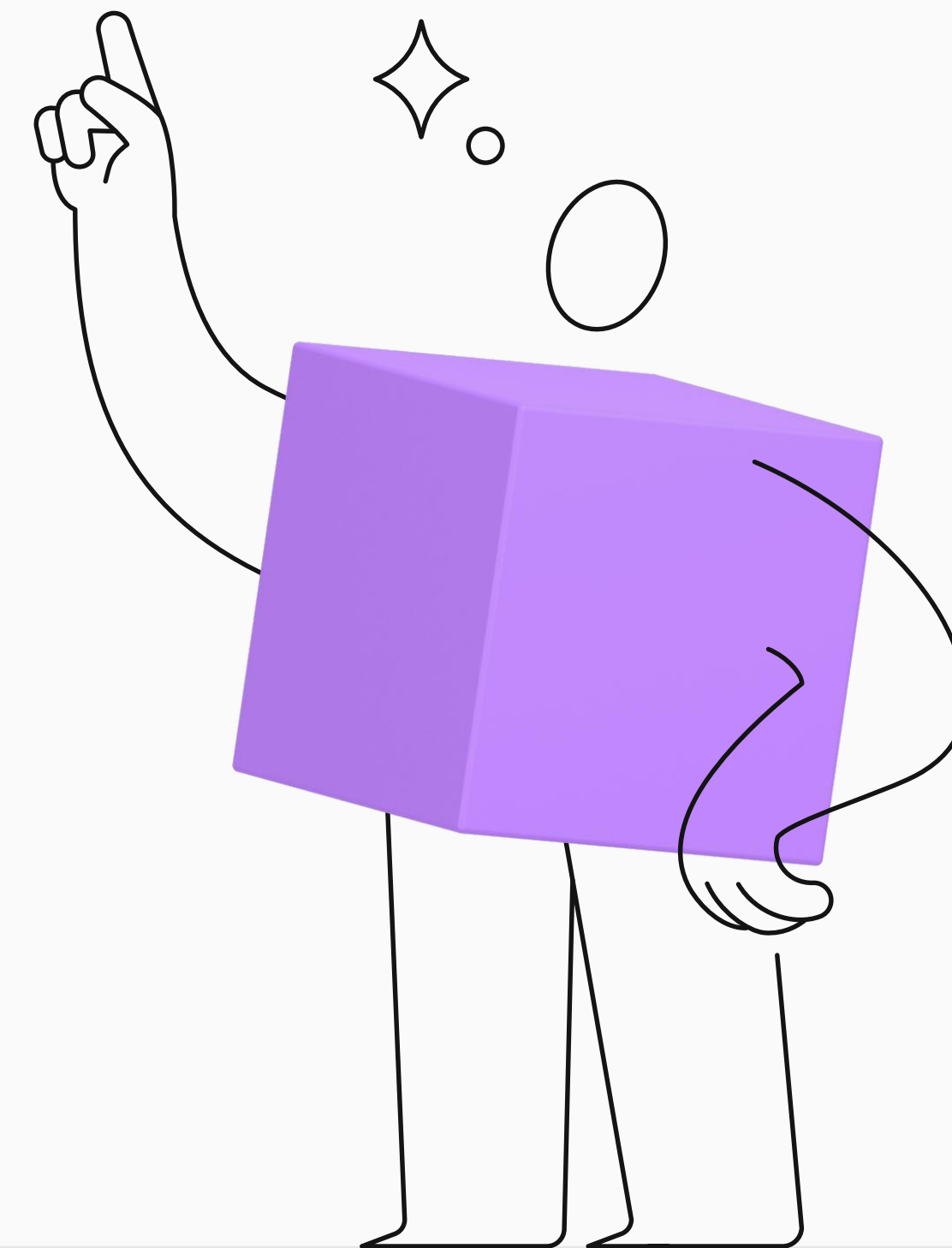
```
CREATE TABLE events_2024_01 PARTITION OF events  
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');  
CREATE TABLE events_2024_02 PARTITION OF events  
FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');
```



Партицирование — это НЕ масштабирование записи!

Важное ограничение

Все партии лежат на одном и том же сервере и используют его ресурсы (CPU, RAM, Disk).

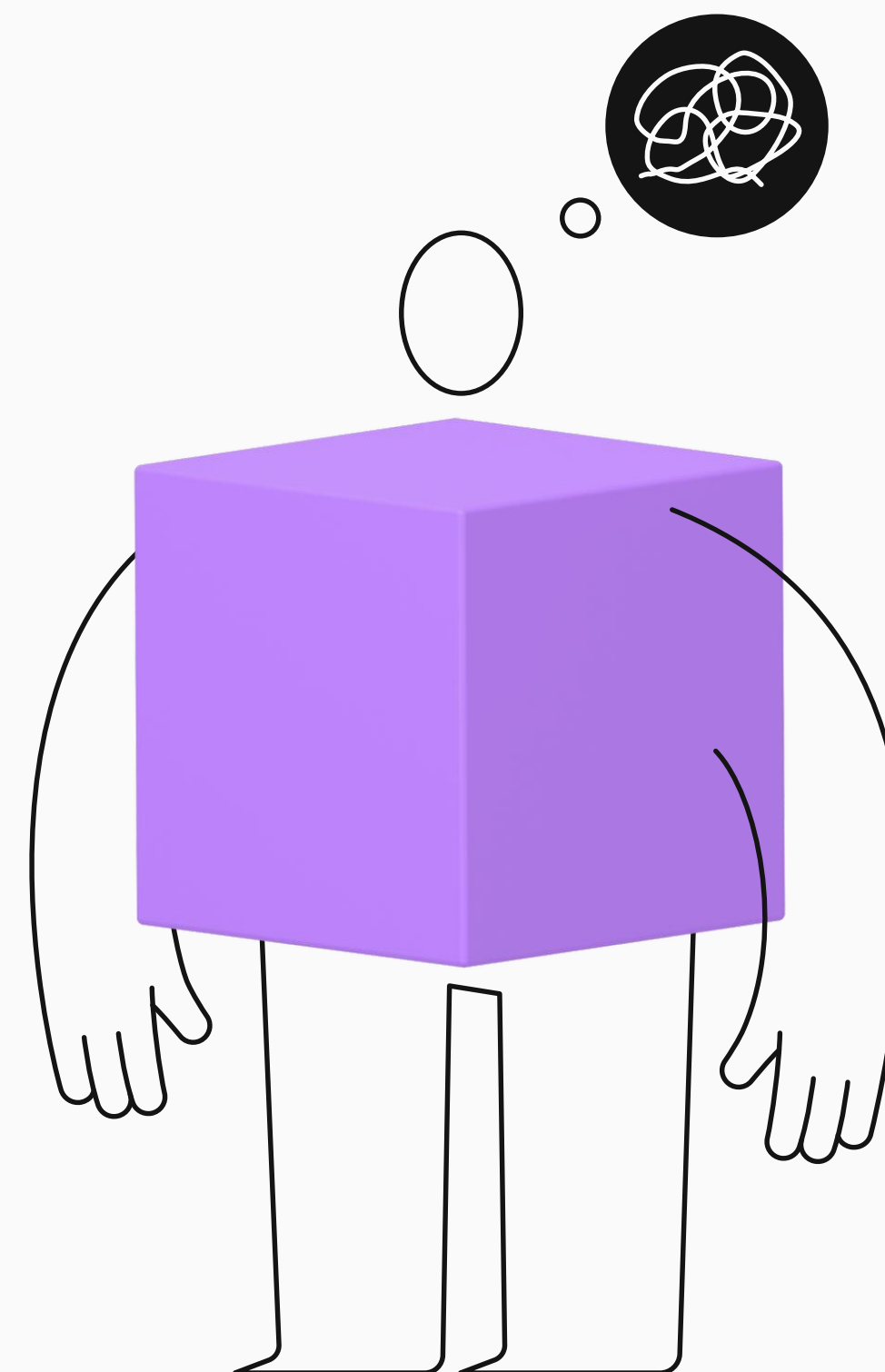


«Чёрный ящик»

ПРОБЛЕМА

Как понять, что база данных «заболела»?

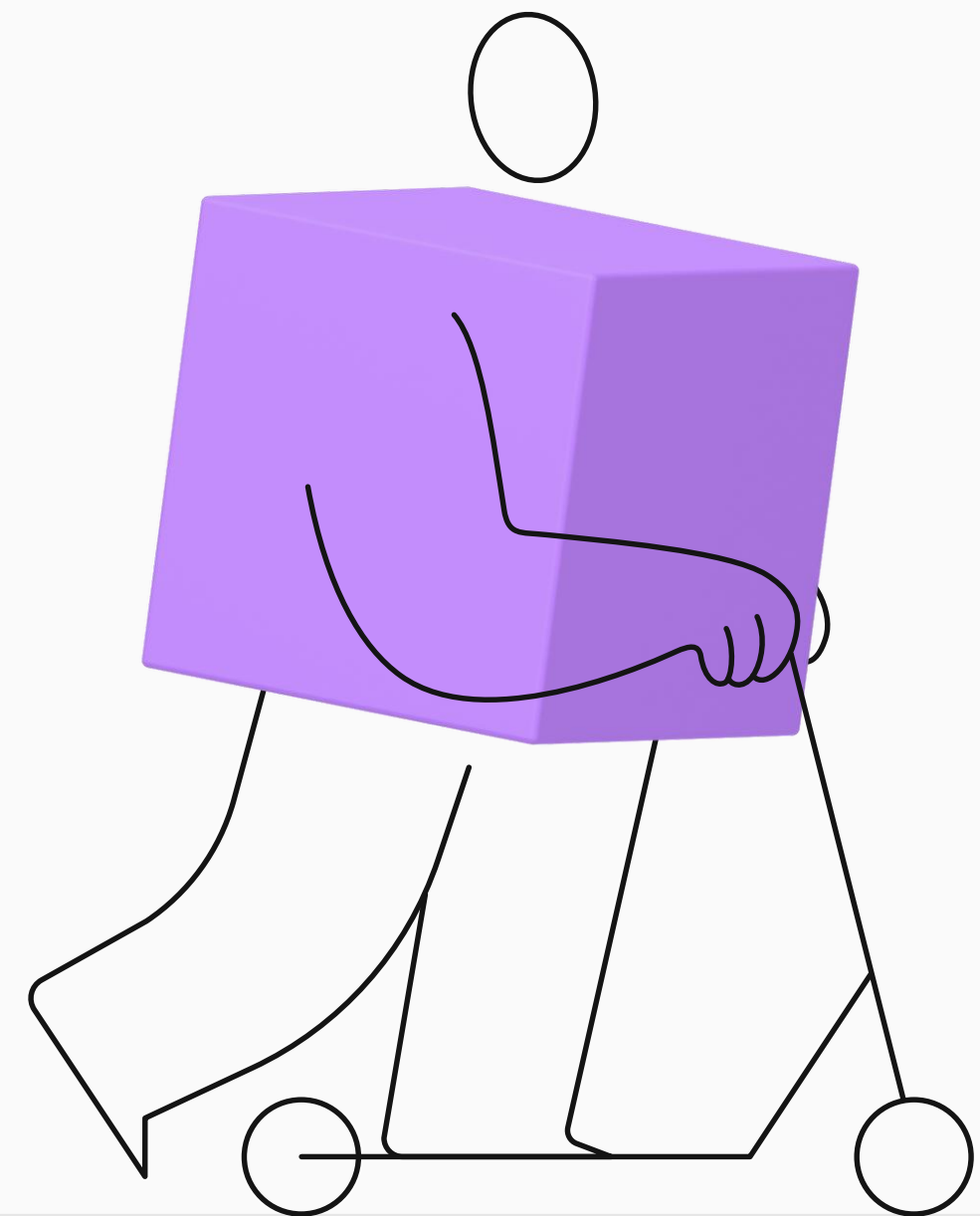
- Почему приложение начало тормозить?
- Какой SQL-запрос «съедает» все ресурсы?
- Сколько сейчас активных подключений?
- Эффективно ли мы используем кеш?
- Не заканчивается ли место на диске?



Статистические представления PostgreSQL

Ключевые представления

- `pg_stat_activity`
- `pg_stat_database`
- `pg_stat_user_tables`



pg_stat_statements

Главный инструмент разработчика

query	calls	total_exec_time	mean_exec_time	rows
SELECT * FROM users WHERE id = \$1	1 500 000	3000	3000	1 500 000
SELECT ... FROM orders JOIN ...	150	150000	1000.0	75000

Настройки в postgresql.conf

```
postgres=#
postgres=# SHOW log_min_duration_statement;
log_min_duration_statement
-----
-1
(1 row)

postgres=# SHOW log_statement;
log_statement
-----
none
(1 row)

postgres=# SHOW log_duration;
log_duration
-----
off
(1 row)

postgres=# SHOW shared_preload_libraries;
shared_preload_libraries
-----

(1 row)
```

```
# Basic configuration for slow query logs.
log_min_duration_statement = 1000 # Log SQL statements with e
log_statement='all '           # Record all SQL statemen
log_duration=on # Record the execution time of each statemen
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,clie

# Configuration of log details.
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits=on # Record lock wait
log_temp_files = 0 # Record temporary file u
log_autovacuum_min_duration = 0 # Record autovacuum oper

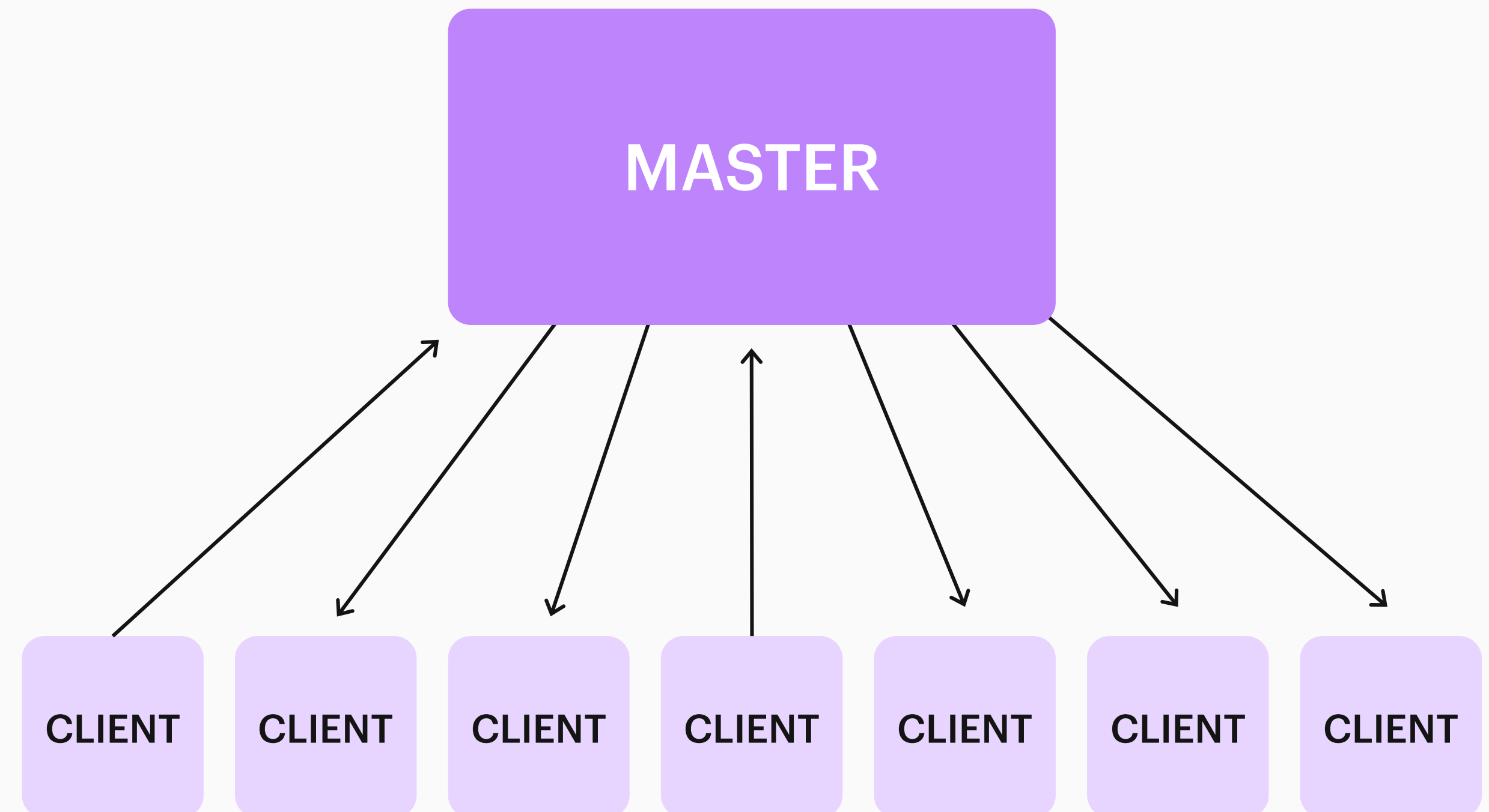
# Log file management.
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_file_mode = 0600
log_rotation_age = 1d
log_rotation_size = 100MB
```


Так где же предел?

«Стеклянный потолок» Single-Master-архитектуры

Предел наступает в следующих случаях.

- 01 Нагрузка на запись превышает возможности одного сервера.
- 02 Объём данных превышает возможности одного сервера.
- 03 Нужна геораспределённая запись с низкой задержкой.

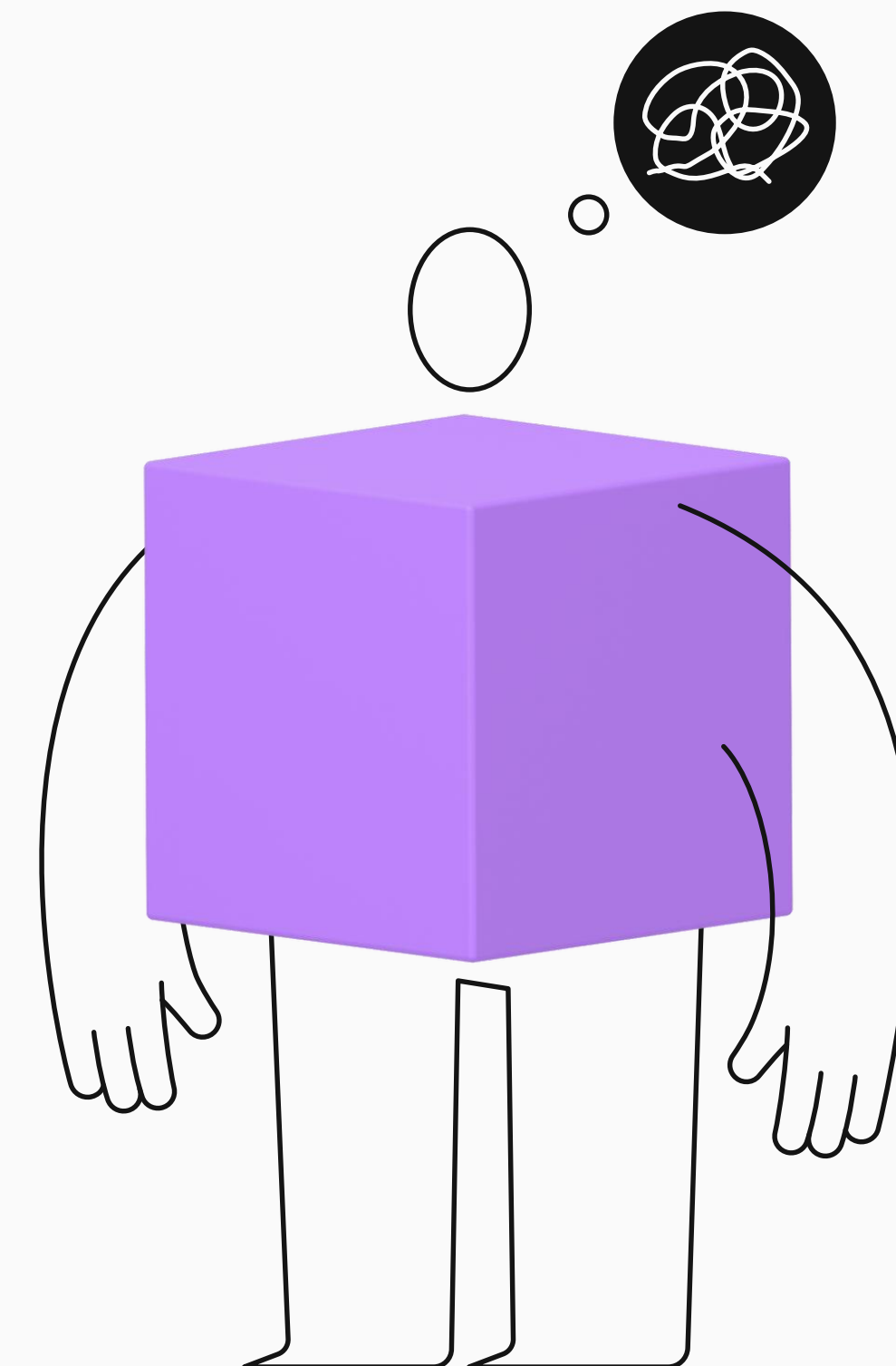


«Ручное» горизонтальное масштабирование

Данные делятся на шарды,
приложение само решает, куда идти.

ПРОБЛЕМЫ

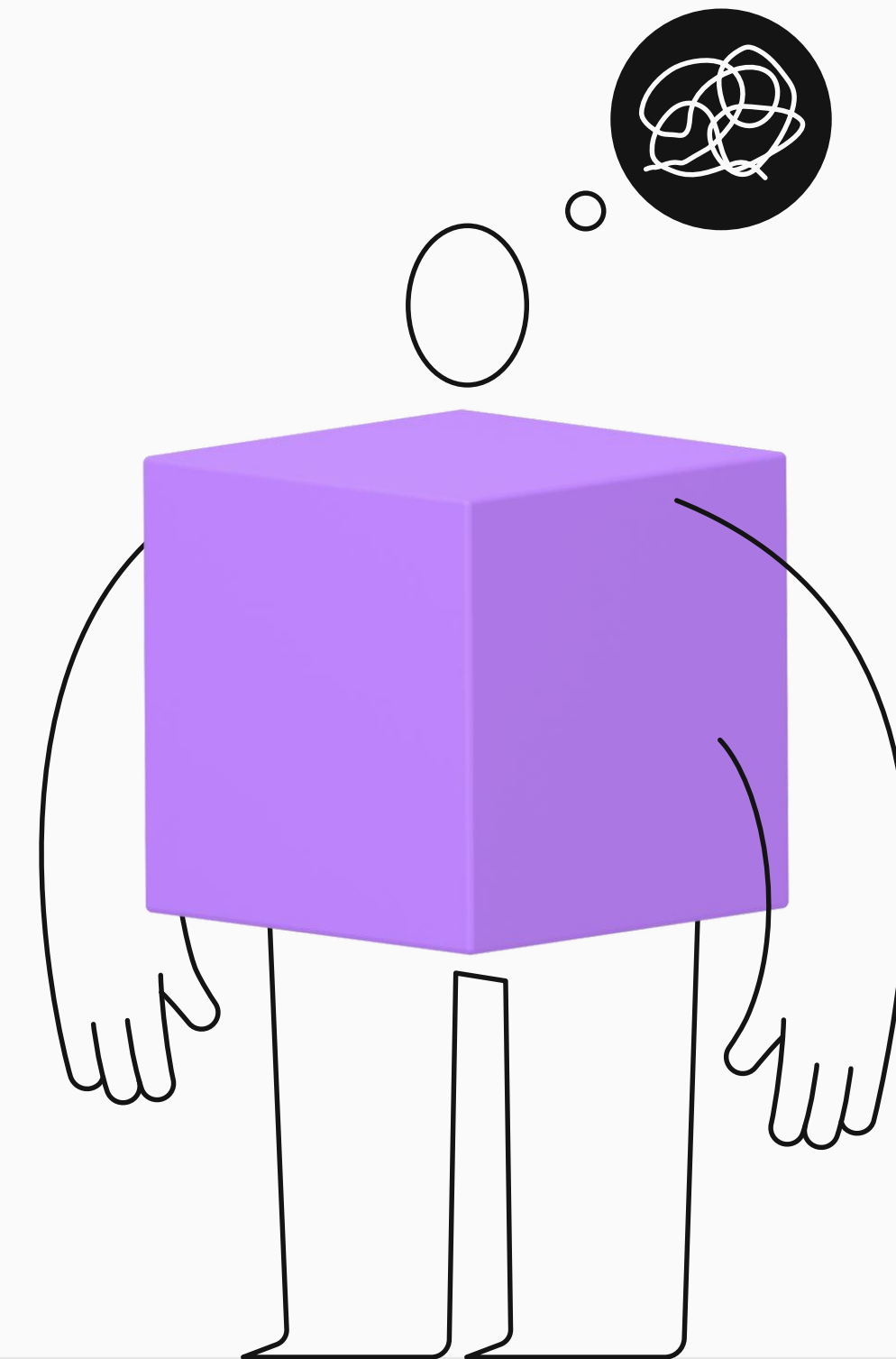
- Невероятно сложно в реализации
- JOIN между шардами — плохая идея
- Сложная перебалансировка
- Нет распределённых транзакций



Что, если бы база данных сама умела это делать?

Требования к «идеальной» распределённой базе

- Multi-Master/Leaderless
- Автоматическое шардирование
- Эластичность
- Встроенная отказоустойчивость



Выводы для разработчика



→ Используйте Patroni для HA

→ Понимайте разницу между балансировкой пулов и парсингом SQL

→ Используйте реплики для чтения

→ Мониторьте `pg_stat_statements`

→ Подумайте о партиционировании заранее

Вопросы

